arXiv:2611.12650v1 [cs.GR] 9 Nov 2026

Explicit LOD-Conditioned Anisotropic Filtering for Neural Texture Compression

Shin Fujieda
Advanced Micro Devices, Inc.
Tokyo, Japan
Shin.Fujieda@amd.com

Takahiro Harada
Advanced Micro Devices, Inc.
Santa Clara, USA
Takahiro.Harada@amd.com

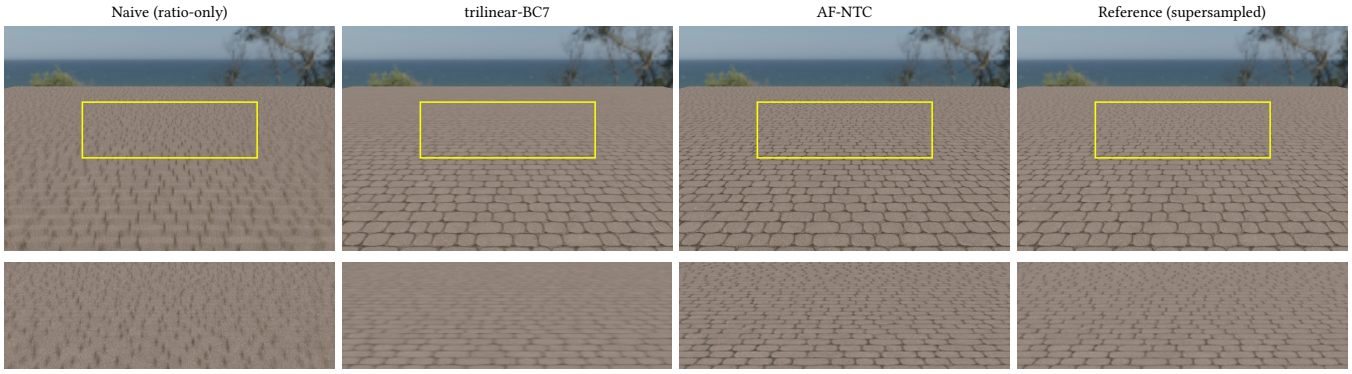


Figure 1: AF-NTC adds explicit anisotropic filtering to a feature-pyramid decoder at 25% of the BC7 budget (`pavement` plane; render vs. a supersampled-EWA reference; bottom rows zooms the far parts). Our *naive* decoder sets the filter from the aspect ratio alone, so it is too blurry even near the camera. The no-AF *trilinear-BC7* baseline, at 4× our size, is too blurry far away. *AF-NTC* is sharp near the camera and blurry far away, and is the closest of the three to the *reference*.

Abstract

Neural texture compression (NTC) stores material textures compactly, but the dominant feature-pyramid design receives no filtering from the GPU texture unit: it learns each material at a fixed UV and level of detail and has no explicit anisotropic-filtering (AF) mechanism. Setting the filter from the footprint aspect ratio alone is not enough: a single input cannot carry both the shape of the footprint and its size, so the image is too blurry near the camera by the same amount at every distance. Our method, AF-NTC, instead conditions the decoder on the same three bounded values a hardware sampler uses: minor-axis scale (LOD), aspect ratio, and major-axis orientation. These drive a tap line along the major axis and a level blend across the minor axis, and we train against anisotropically filtered ground truth. At 25% of a full BC7 mip chain, AF-NTC beats the no-AF trilinear-BC7 baseline on all ten material–scene pairs we compare (+1.3 to +7.6 dB), and it is 0.0 to 3.4 dB below the 4×-larger anisotropic-BC7, depending on the material.

CCS Concepts

• **Computing methodologies** → **Ray tracing**; **Neural networks**; **Image compression**.

Keywords

Texture Compression, Neural Networks, Anisotropic Filtering

1 Introduction

Neural texture compression (NTC) aims to store material textures below the on-disk size of fixed-rate block compression (BC1–BC7 depending on texture types), at equal or better quality [Microsoft 2020; Vaidyanathan et al. 2023]. A texture is sampled many times during rendering under minification and at grazing angles, so anisotropic filtering (AF) is necessary: it keeps detail near the camera and correctly removes it with distance. NTC must therefore be evaluated not only on compressed size but also on filtering quality.

Two families of NTC handle AF in opposite ways. *Latent-in-BC* methods store learned features inside BC textures and let the GPU texture unit filter the latents in hardware, so AF is provided by the hardware sampler at no additional cost [Belcour and Benyoub 2025; Weinreich et al. 2024]. *Feature-pyramid* NTC instead learns multi-resolution feature grids with a small multilayer perceptron (MLP) decoder [Vaidyanathan et al. 2023]. It has no hardware filtering and no explicit anisotropic filtering mechanism, so it aliases at one sample per pixel and relies on stochastic texture filtering (STF) [Pharr et al. 2024]. This work makes AF an *explicit* operation in a feature-pyramid decoder, conditioned on the pixel footprint and trained against anisotropically filtered ground truth, so that the result is sharp near the camera and blurry far away, which a naive tap blend cannot do. We call this decoder AF-NTC.

Because the aspect ratio alone cannot carry the footprint's size, we instead condition the decoder on the same bounded (`rot`, `stretch`, LOD) triple that hardware AF uses (i.e., minor-axis scale (level of detail (LOD)), aspect ratio (`stretch`), and major-axis orientation (`rot`)), and drive it along two paths: a major-axis tap line and a

minor-axis level blend. On the pavement scene in Fig. 1, AF-NTC is sharp near the camera and blurry far away, and stays closer to a supersampled reference than the no-AF baseline.

Our contributions are:

- **Diagnosis and a minimal fix.** The aspect ratio alone cannot set the filter; the bounded (rot, stretch, LOD) triple restores the quality.
- **Two-axis learned filter.** For a decoder without hardware filtering, we combine a major-axis tap-line length with a minor-axis level blend.
- **Compression vs. filtering.** An ablation on model size shows which part of the error is compression and which is filtering.

2 Related Work

Feature-pyramid NTC. The dominant NTC design stores a material as a stack of learnable multi-resolution feature grids plus a small MLP decoder, and reports state-of-the-art quality at a matched on-disk size [Vaidyanathan et al. 2023]. NTBC [Fujieda and Harada 2024] follows the same idea but packs the neural features into BC blocks so that a standard asset pipeline can carry them. These decoders receive no filtering from the GPU texture unit. They learn the material at fixed UV and LOD, so one sample per pixel aliases, and practical systems rely on STF, which samples a random subset of taps, reducing temporal aliasing at the cost of added noise [Pharr et al. 2024]. Vaidyanathan et al. [2023] report that software AF requires too many taps for real time. We target this family.

Latent-in-BC NTC. A second family stores latent feature vectors inside BC textures, lets the GPU sampler filter the *latents* in hardware, and then decodes with a tiny MLP [Belcour and Benyoub 2025; Weinreich et al. 2024]. Anisotropic filtering is then effectively *free*: no filtering logic is added to the decoder, a property we call *free AF*. A network trained only on isotropic samples generalizes to well-filtered output, because the sampler gives the actual, bounded footprint, and AF is just a weighted blend of taps over that footprint. This is the property a feature-pyramid decoder lacks: it sees only UV, so the footprint must be an *explicit, learned* input.

Hardware AF, EWA, and the footprint. Classic high-quality filtering, such as the elliptical weighted average (EWA) [Heckbert 1989], integrates the texture over the elliptical footprint of a pixel. Approximated hardware AF samples several trilinear taps along the footprint's major axis over a mip pyramid [McCormack et al. 1999; Williams 1983]. Hardware AF derives the footprint ellipse from the screen-to-UV Jacobian; we give the derivation and the bounded inputs the sampler uses in Sec. 3.2.

3 Method

AF-NTC makes AF explicit inside the feature-pyramid decoder. Fig. 2 shows the pipeline of our proposed method.

3.1 Failure of Ratio-only Conditioning

The naive anisotropy-aware design derives its filter length from stretch alone, e.g., a length of the form

$$\texttt{length} = \texttt{stretch} \cdot k, \tag{1}$$

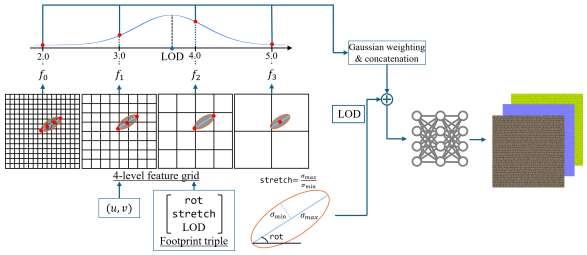


Figure 2: Overview of AF-NTC. The footprint triple sets the tap geometry on each grid level (bottom), the LOD-weighted blend combines levels (top), and only the concatenated feature vector and LOD reach the MLP.

with stretch the ratio of the long to short footprint axis and k a single fixed constant.

This fails because the correct filter is set by two things that vary independently: the footprint's absolute size (how much to blur (i.e., mip level)) and its aspect ratio (how elongated it is). stretch encodes only the ratio, so a ratio-only decoder must apply a fixed size k that is correct at only one distance; it can never represent a near and grazing footprint, which is small yet strongly elongated. The *Naive* column of Fig. 1 shows the result: the image is too blurry near the camera, by the same amount at every distance. The missing quantity, absolute size, must therefore be an input of its own, which motivates the bounded (rot, stretch, LOD) triple.

3.2 Bounded Hardware-AF Footprint Triple

From the screen-to-UV Jacobian J, we form the metric $A = J J^\top$, whose eigenvalues are $\sigma_{\max}^2 \geq \sigma_{\min}^2$; their square roots $\sigma_{\max} \geq \sigma_{\min}$ are the major- and minor-axis lengths, and the major eigenvector gives the orientation. From these, we form the triple as follows:

$$\text{LOD} = \log_2(\sigma_{\min} \cdot \texttt{tex_res}), \qquad \in [0, 12] \text{ at 4K}, \tag{2}$$

$$\texttt{stretch} = \sigma_{\max}/\sigma_{\min}, \qquad \in [1, 16], \tag{3}$$

$$\texttt{rot} = \angle(\text{major axis}), \qquad \in [0, \pi). \tag{4}$$

$\sigma_{\min}$ sets the LOD level, stretch sets the tap count and, together with $\sigma_{\min}$, the tap-line length, and rot gives the major-axis orientation. The renderer supplies LOD per pixel; from it, we recover the minor-axis length by inverting Eq. 2:

$$\sigma_{\min} = 2^{\text{LOD}} / \texttt{tex_res}. \tag{5}$$

LOD and $\sigma_{\min}$ should be consistent, and we use this single recovered $\sigma_{\min}$ everywhere it is needed: both to place the taps (Eq. 6) and to build the anisotropically filtered ground truth (Sec. 3.4). An elliptical footprint is fully described by its size, elongation, and orientation, and the triple (rot, stretch, LOD) gives exactly these three.

3.3 Two-Axis Learned Filter

The decoder reads input features from a four-level feature pyramid of 2D grids, each storing a 12d feature, bilinearly sampled on each tap and concatenated across levels into a 48d grid-feature vector. The footprint triple conditions the decoder both geometrically and as explicit inputs. Geometrically, as shown in Fig. 2, it sets how the grids are filtered along the two footprint axes: rot orients the

major axis, $\sigma_{\min}$, which is recovered from LOD with Eq. 5, and `stretch` together set the major-axis tap-line length (Eq. 6), and LOD selects the mip level and weights that the minor-axis level blend. As explicit inputs, the 48d grid-feature vector is concatenated with the single normalized scalar LOD/`lod_max`, forming the 49d vector that feeds the MLP; `rot` and `stretch` are used only through the tap geometry, as tap orientation and tap-line length, not as MLP inputs. An ablation in Appendix B shows that also feeding (`rot`, `stretch`) to the MLP as an 18d positional encoding changes quality only slightly, because the tap geometry already uses `rot` and `stretch`.

Major axis: absolute tap-line length. The major-axis tap-line length is set from the *absolute* minor-axis length, not the ratio:

$$\texttt{length_major} = \sigma_{\min} \cdot \texttt{stretch} \quad (= \sigma_{\max}), \tag{6}$$

with $\sigma_{\min}$ from Eq. 5. Unlike the naive form of Eq. 1, where `stretch` is multiplied by the fixed constant k, here it is multiplied by the absolute length $\sigma_{\min}$, so the tap line spans the true major-axis length $\sigma_{\max}$ rather than a distance-independent length. Taps are placed along the major axis, one budget per grid level, with fixed counts $[16, 8, 4, 2]$ (30 taps in total) from the finest to the coarsest level. We choose this 30-tap budget to match hardware-AF tap counts and lower decode cost rather than for peak quality.

Minor axis: level blend. Taps along the major axis alone leave the minor axis unfiltered. We band-limit the minor axis with an LOD-weighted blend across the four grid levels. Each level l has an LOD center,

$$c_l = \log_2(\texttt{tex_res}/\texttt{res}_l), \tag{7}$$

and its 12d feature is weighted by a Gaussian in LOD about its center with $\sigma = 0.7$. Then, the weighted features are concatenated, keeping the grid-feature width fixed at 48.

3.4 Training, Architecture, and Model Size

Ground truth. The training target *simulates* hardware AF instead of computing an exact EWA integral: we place $\lceil \texttt{stretch} \rceil$ taps along the major axis over a length of $(2^{\text{LOD}}/\texttt{tex_res}) \cdot \texttt{stretch}$, and fetch each tap trilinearly at mip = LOD, which filters the minor axis. It is therefore an approximation: it agrees with a supersampled EWA reference to 37–41 dB PSNR on our scenes. We use it only for training and measure quality on the rendered image (Sec. 4). Note that the decoder's tap budget $[16, 8, 4, 2]$ is fixed, while this target uses $\lceil \texttt{stretch} \rceil$ taps.

Filter-parameter sampling. Instead of generating training footprints from rendered views, we sample the (`rot`, `stretch`, LOD) triple directly from parameter space, independently of one another. `rot` and `stretch` are sampled uniformly over their ranges, and LOD is sampled area-proportionally, $p(\text{LOD}) \propto 4^{-\text{LOD}}$, so that coverage matches the screen area each LOD occupies. Independent sampling gives the decoder footprint size and ratio as uncorrelated inputs, so it sees combinations that no single view provides.

Training. As in other NTC work, our model is fitted to one 4K material and shipped with it, so training and evaluation share that texture set. The model is implemented, trained, and evaluated in PyTorch. We train our model with an L_2 loss, with optional LOD-weighting and the normal-map channels up-weighted. We then quantize to int8 by quantization-aware training (QAT) with the Brevitas quantization library [Franco et al. 2026].

Architecture and model size. The encoder holds a four-level 2D feature grid at resolutions 1024/512/256/128, so the per-level LOD centers c_l in Eq. 7 are 2, 3, 4, 5. The decoder is an MLP that maps the 49d conditioned feature vector through hidden widths $[192, 192]$, leaky-ReLU activations, and a final sigmoid, outputting 9 channels: albedo (3), an ARM channel packing ambient occlusion, roughness, and metallic into one texture (3), and a normal (3). AF-NTC has an on-disk size of 16.75 MB (int8-packed grids and MLP), about 25% of the 67 MB that these same three 4K maps (albedo, ARM, and normal) occupy as a full BC7 mip chain.

4 Results

Setup. All our numbers are PSNR measured in *rendering space* rather than texture space: we render with the maps that each method produces and compare against a supersampled EWA ground truth rendered under the same lighting, so the two images differ only in the maps. We evaluate five 4K materials, ordered from low to high spatial frequency in Table 1, each rendered in two scenes: a flat `plane` (sharp near, blurred far) and a curved `sphere`. We compare AF-NTC against two BC7 baselines that use the same software AF implementation as AF-NTC and store the full mip chain, about 4× the AF-NTC size: *aniso-BC7* (full anisotropic filtering, a strong upper reference) and *trilinear-BC7* (the same BC7 maps filtered only isotropically, with trilinear mip sampling and no AF, a weaker baseline). Unless noted, we default to the int8 model (Sec. 3.4).

Conditioning. Table 1 builds up AF-NTC one part at a time on top of an *isotropic* baseline: first training on anisotropic ground truth (*+aniso*), then feeding LOD to the decoder (AF-NTC). The isotropic baseline follows the free-AF approach [Belcour and Benyoub 2025; Weinreich et al. 2024]: the same decoder trained on isotropic samples only, with no footprint input. Learning the AF footprint explicitly improves it throughout: each part raises PSNR on all materials and both scenes. The gain is smallest on low-frequency `blue_metal` (+1.02 dB plane) and largest on the high-frequency materials, up to +4.66 dB (plane, `stone_floor`) and +5.85 dB (sphere, `pavement`).

Comparison. The same table places AF-NTC, at 25% of the BC7 mip-chain budget, between the two BC7 baselines. AF-NTC beats trilinear-BC7 on all ten material–scene rows, by +1.31 to +7.63 dB, including the highest-frequency `stone_wall`. It stays below aniso-BC7 on every row, by 0.03 to 3.37 dB (mean 1.57): closest on `stone_floor` and furthest on `pavement` plane. Renders and error maps for all ten rows are in Appendix D (Fig. 6).

Meshes. To test AF-NTC on real 3D models, not just the flat and curved test surfaces, we train it per asset on two PolyHaven meshes whose 4K maps are non-stationary, with a unique UV layout rather than a tiling pattern: a concrete `cat` statue and a wine `barrel`. Table 2 shows PSNR in rendering space on each asset, and Fig. 3 shows the `cat` images. At 25% of the BC7 size, AF-NTC stays just below the full-4K aniso-BC7 reference by at most 0.07 dB on the two assets and beats the no-AF trilinear-BC7 by +2.0 (`cat`) to +3.5 dB (`barrel`). Against the equal-byte, half-resolution (2K) aniso-BC7, AF-NTC is +0.28 dB on `cat` and −0.07 dB on `barrel`. On both assets,

Table 1: AF-NTC, its ablation, and the two BC7 baselines (PSNR, dB). *iso* and *+aniso* are the same decoder trained on isotropic and on anisotropically filtered ground truth; AF-NTC adds LOD conditioning. *aniso* and *tri* are aniso-BC7 and trilinear-BC7, which store the full mip chain at 4× the AF-NTC size. Materials run from low to high content frequency.

		ours (ablation)			BC7, 4× size	
Material	Scene	iso	+aniso	AF-NTC	aniso	tri
`blue_metal`	plane	50.54	51.27	51.56	53.62	45.87
	sphere	50.12	51.63	51.93	53.01	47.45
`clay_roof`	plane	33.65	34.67	35.02	35.57	27.39
	sphere	37.63	39.33	39.76	41.42	35.80
`stone_floor`	plane	33.94	36.71	38.60	39.08	35.56
	sphere	34.01	37.36	39.08	39.11	34.83
`pavement`	plane	33.64	36.27	37.47	40.84	36.11
	sphere	33.81	37.54	39.66	41.78	37.70
`stone_wall`	plane	30.27	32.68	33.48	35.09	30.21
	sphere	30.18	33.45	35.48	38.24	34.17

Table 2: AF-NTC on mesh assets (PSNR, dB). *aniso* and *tri* are the full-4K BC7 references. *aniso-2K* is the equal-byte 2K aniso-BC7.

Asset	AF-NTC	aniso	tri	aniso-2K
`cat`	40.34	40.40	38.33	40.06
`barrel`	46.02	46.09	42.54	46.09

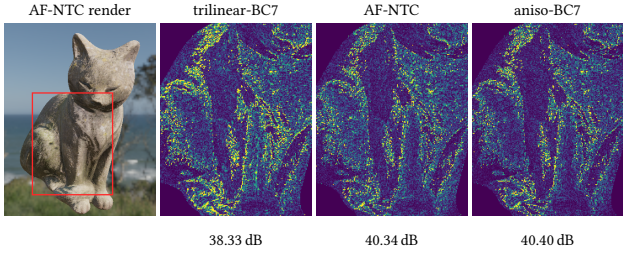


Figure 3: `cat` mesh. *Left*: the AF-NTC render; the red box marks the inset. *Right*: ×16 absolute error against the super-sampled reference over that inset, with whole-frame PSNR. Trilinear-BC7 leaves clearly more error, while AF-NTC and the 4×-larger aniso-BC7 are within 0.1 dB.

the footprints are large (LOD ≈1.4–2.4), so AF already removes most fine detail and a 2K map loses little; we do not claim this equal-byte result for the finer footprints of Table 1, where the 2K chain is 0.4 to 3.3 dB better than AF-NTC. The `barrel` images are in Appendix C (Fig. 5).

Ablations. Two ablations on `pavement` and `stone_floor` show where the error comes from; the table and discussion are in Appendix A (Table 3). Removing the minor-axis level blend costs 5.3 to 9.3 dB, most of it near the camera where footprints are smallest, so the blend is the most important part of the filter. A ~4× larger model gains at most 0.4 dB: at this budget, the error left is set by compression, not by model size.

5 Conclusion and Discussion

We showed that a feature-pyramid NTC decoder can filter anisotropically when given explicit control: a bounded (`rot`, `stretch`, LOD) triple plus a two-axis filter—a tap line along the major axis and a level blend across the minor axis.

Unlike stochastic filtering [Pharr et al. 2024], which keeps one decode per pixel but trades aliasing for noise that a temporal post-process must resolve, our filter is deterministic. A single decode returns a noise-free filtered value that needs no post-processing and is therefore easy to integrate into an existing pipeline, at the cost of more taps per decode.

Scope and limitations. We claim correct filtering at a fixed compression budget, not a general size–quality win, and we do not claim to match BC7. AF-NTC comes within 0.5 dB of the 4×-larger aniso-BC7 on `stone_floor`, but stays below it on every row, by up to 3.4 dB (Table 1). We report no runtime evaluation: our AF is a PyTorch software emulation, not an optimised GPU implementation on a texture sampler. How that tap budget trades quality against decoding performance, measured on real hardware, is future work.

References

Laurent Belcour and Anis Benyoub. 2025. Hardware Accelerated Neural Block Texture Compression with Cooperative Vectors. In *High-Performance Graphics - Symposium Papers*, Aaron Knoll and Christoph Peters (Eds.). The Eurographics Association. doi:10.2312/hpg.20251173

Giuseppe Franco, Pablo Monteagudo-Lago, Ian Colbert, Alessandro Pappalardo, and Nicholas J Fraser. 2026. *Xilinx/brevitas*. doi:10.5281/zenodo.3333552

Shin Fujieda and Takahiro Harada. 2024. Neural Texture Block Compression. *Workshop on Material Appearance Modeling* (2024). doi:10.2312/MAM.20241178

Paul S. Heckbert. 1989. *Fundamentals of Texture Mapping and Image Warping*. Technical Report UCB/CSD-89-516. http://www2.eecs.berkeley.edu/Pubs/TechRpts/1989/5504.html

Joel McCormack, Ronald Perry, Keith I. Farkas, and Norman P. Jouppi. 1999. Feline: Fast Elliptical Lines for Anisotropic Texture Mapping. In *Proceedings of SIGGRAPH*. ACM Press/Addison-Wesley Publishing Co., USA, 243–250. doi:10.1145/311535.311562

Microsoft. 2020. Texture Block Compression in Direct3D 11. https://learn.microsoft.com/en-us/windows/win32/direct3d11/texture-block-compression-in-direct3d-11.

Matt Pharr, Bartlomiej Wronski, Marco Salvi, and Marcos Fajardo. 2024. Filtering After Shading With Stochastic Texture Filtering. *Proc. ACM Comput. Graph. Interact. Tech.* 7, 1, Article 14 (May 2024), 20 pages. doi:10.1145/3651293

Karthik Vaidyanathan, Marco Salvi, Bartlomiej Wronski, Tomas Akenine-Moller, Pontus Ebelin, and Aaron Lefohn. 2023. Random-Access Neural Compression of Material Textures. *ACM Trans. Graph.* 42, 4, Article 88 (July 2023), 25 pages. doi:10.1145/3592407

Clément Weinreich, Louis de Oliveira, Antoine Houdard, and Georges Nader. 2024. Real-Time Neural Materials using Block-Compressed Features. *Computer Graphics Forum* 43, 2 (2024), e15013. doi:10.1111/cgf.15013

Lance Williams. 1983. Pyramidal Parametrics. *SIGGRAPH Comput. Graph.* 17, 3 (1983), 1–11. doi:10.1145/964967.801126

A Combined Ablations

All numbers in this appendix are PSNR in rendering space, as in Sec. 4; higher is better. Every AF-NTC number is an average over several training runs that differ only in their random seed.

Table 3 gives the filtering and model-size ablations. Its *AF-NTC* column is our default model, so it repeats the AF-NTC numbers of Table 1.

Filtering. Our filter has two parts: a tap line along the major axis and a level blend across the minor axis. The *no-blend* column keeps the tap line and removes the blend, so it still filters along the major axis but no longer across it. The blend is the more important of the two: without it, PSNR drops by 5.3 to 9.3 dB. Fig. 4 shows where

Table 3: Combined ablations (dB). *no-blend* removes the minor-axis level blend from AF-NTC; *2× feat.* uses 24d per-level features (vs. 12d), and *2× res.* uses higher-resolution grids (~4× params).

Material	Scene	no-blend	AF-NTC	2× feat.	2× res.
`pavement`	plane	32.16	37.47	37.45	37.63
	sphere	30.36	39.66	39.67	39.83
`stone_floor`	plane	31.91	38.60	38.52	38.93
	sphere	30.13	39.08	39.18	39.29

that error is. Near the camera, the footprints are small, and the decoder has to read the grids at a fine level; without the blend, it cannot, and the pattern is washed out. Far from the camera, where a coarse level is the right one anyway, the two look the same.

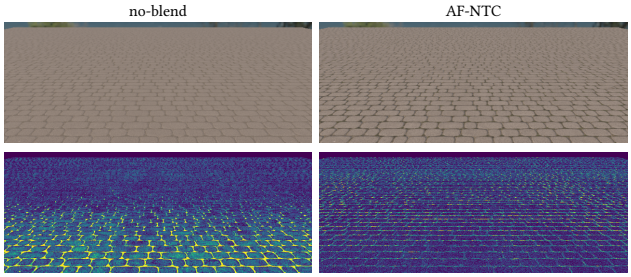


Figure 4: Effect of the minor-axis level blend (`pavement` plane). *Top*: rendered crop. *Bottom*: ×16 absolute error against the supersampled reference. Without the blend, the decoder cannot read the grids at a fine enough level for the small footprints near the camera, so the pattern there is washed out. Further away, the two are almost the same.

Model size. We made the model larger in two ways: doubling the feature dimension and using higher-resolution grids. Both change PSNR by at most 0.4 dB (Table 3). The error left at this budget is thus set by compression, not by model size: a larger model cannot bring back detail that the footprint filter has already averaged away.

B Feeding (`rot`, `stretch`) to the MLP

Our decoder gives the multilayer perceptron (MLP) only the level of detail (LOD). `rot` and `stretch` reach it another way: they place the taps, as their orientation and the length of the tap line. Table 4 asks whether the MLP should be given them as an additional 18d positional encoding. It should not: the two differ by at most 0.05 dB, so we keep the smaller input.

Table 4: Giving (`rot`, `stretch`) to the MLP, on `pavement`. Ours takes a 49d input, the 48d grid features plus LOD; the 67d variant adds an 18d positional encoding of (`rot`, `stretch`).

Scene	49d (ours)	67d (+PE)
plane	37.53	37.48
sphere	39.74	39.71

C Mesh Assets

Fig. 5 shows the `barrel` asset in Table 2. The inset covers the side of the barrel, where the surface turns away from the camera, and the footprints are the most stretched. There, trilinear-BC7 leaves clearly more error on the wooden planks and the metal rings, while AF-NTC and aniso-BC7 look the same.

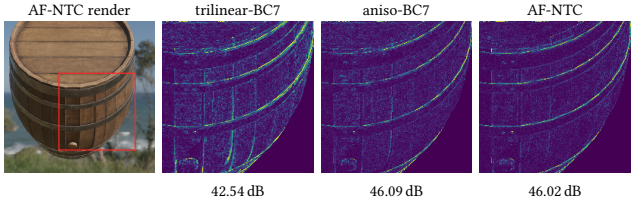


Figure 5: `barrel` mesh. *Left*: AF-NTC render, with the inset region marked in red. *Right*: ×16 absolute error against the supersampled reference for each method over that inset; numbers are whole-frame PSNR. Over this inset, AF-NTC and aniso-BC7 are within 0.1 dB of each other.

D All Ten Material–Scene Pairs

Sec. 4 shows only one of the ten material–scene rows of Table 1 as a picture, the `pavement` plane in the teaser (Fig. 1); Fig. 6 shows all ten. Trilinear-BC7 blurs the surface pattern away wherever the footprint is stretched, most visibly on the `clay_roof` plane, the row where AF-NTC gains the most over it (+7.6 dB). AF-NTC keeps the pattern, and on most rows its error is close to that of the 4×-larger aniso-BC7; it is worse only on the `pavement` plane (−3.4 dB) and the `stone_wall` sphere (−2.8 dB), and even there the two stay visually close.

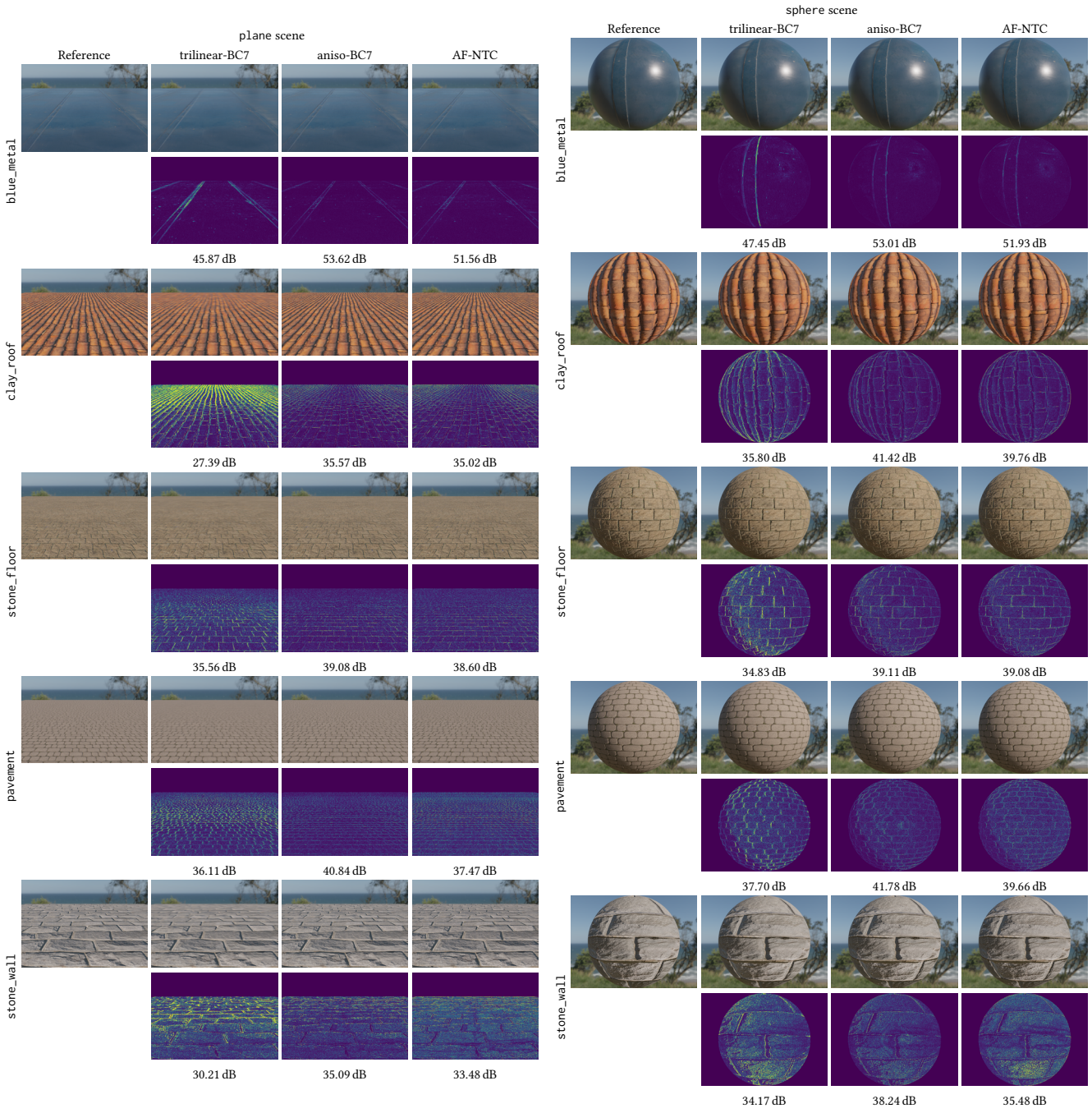


Figure 6: All ten material–scene pairs: the `plane` scene on the left, the `sphere` scene on the right, five materials each, ordered from low to high content frequency. Each material has a row of renders and the ×16 absolute error against the supersampled reference for the three compressed methods. Every material scores higher on the sphere than on the plane.